HW #2: Trade Server

COM-3820 Parallel Programming

Background

You will build the brokerage at the core of a stock trade server. I provide the HTTP plumbing; you implement the brokerage behind it. I will grade you on two things: whether your brokerage stays correct when many threads use it at once, and how much work it gets through.

You may use any coding agent. An agent builds what you ask for; knowing what to ask for is your job. I will grade your code as it runs.

The system

Clients hold cash and stock positions at a brokerage, and they buy from and sell to the brokerage's inventory at the current market price. Requests arrive over HTTP and are handled concurrently by a pool of broker threads.

- **Brokers.** `TradeServer` runs an `HttpServer` whose executor is a fixed pool of N threads; each thread is a broker. `java -jar ./target/trade-server-1.0.jar 4` starts 4 brokers.
- **Routes.** `BrokerService` maps `/prices`, `/register`, `/portfolio`, `/marketvalue`, `/buy`, `/sell`, `/swap` and `/transfer` to the `Brokerage`, and maps exceptions to status codes as its Javadoc specifies.
- **Brokerage.** `Brokerage` holds every client's account and the brokerage's own inventory and cash. Each stock starts with the share count in `stocks.csv`.
- **Prices.** `StockPrices` is a singleton whose background thread changes one stock's price every 500 ms.
- **Your code.** You implement `StockImpl`, `PortfolioImpl`, `BrokerageImpl` and `BrokerServiceImpl` in `edu.yu.parallel.Impl`.

Build and run:

```
trade-server> mvn clean package
trade-server> java -jar ./target/trade-server-1.0.jar 4
```

Requirements

Implement the four classes according to the Javadoc. Any number of threads may call any operation at once. Each operation must appear to take effect at a single instant, as if nothing else were running, and every request must eventually complete.

Grading

I will grade you out of 100 points, plus up to 5 bonus.

- **Correctness (70).** The public tests, which you have, cover basic functionality on a single thread; run them with `mvn test`. You should also write your own tests to check that your brokerage stays correct under multithreading. I will test this with my own private tests on my grading machine. I run each test 5 times, and it passes only if all 5 runs pass. If you fail any correctness test, your grade is capped at 80.
- **Throughput (30, plus up to 5 bonus).** I will measure your brokerage's throughput (completed operations per second) with my own load generator on my 20-core grading machine, and score it on a scale from a brokerage with a basic implementation (B = 517,987 ops/s) up to my reference implementation. If you beat my reference, you earn the bonus. I use the median of 3 runs, each in a fresh JVM with `-Xms4g -Xmx4g -XX:+AlwaysPreTouch`.

The load I measure

- 40 threads call your `BrokerageImpl` directly (not over HTTP), as fast as they can.
- 30 stocks and 10,000 clients; some stocks and some clients are far busier than others.
- Mostly buys and sells, with some swaps, transfers, portfolio reads and market-value reads.
- Prices change every millisecond, and `snapshot()` is called every 100 ms.
- At the end, a snapshot must conserve every share and every cent, or the run's throughput counts as 0.

How you load-test your brokerage before you submit is up to you.

Constraints

- Use Java 21, built and run with Maven. Log through the provided Log4j2 logger; do not use `System.out`.
- Do not modify `App`, `TradeServer`, `StockPrices`, `StockReader`, the provided tests, or the POM. You may add methods, fields and classes, but do not change the provided interface signatures.
- Your code **may not use concurrent collections anywhere**: no `ConcurrentHashMap`, `ConcurrentSkipListMap`, `CopyOnWriteArrayList`, blocking queues, or `Collections.synchronizedMap` and its relatives. You may use locks and atomics.
- I have included an optional JDK Flight Recorder guide, `JFR-Intro.md`.

Submission

- All submitted assignments must be checked into the YU GitHub system, into the **PP-COM3820** branch that you created for this course.
- This assignment will be checked into the `/assignment-2` folder. That is, your `pom.xml` file will be in this directory.
- Put all your code in `edu.yu.parallel` or a subpackage.